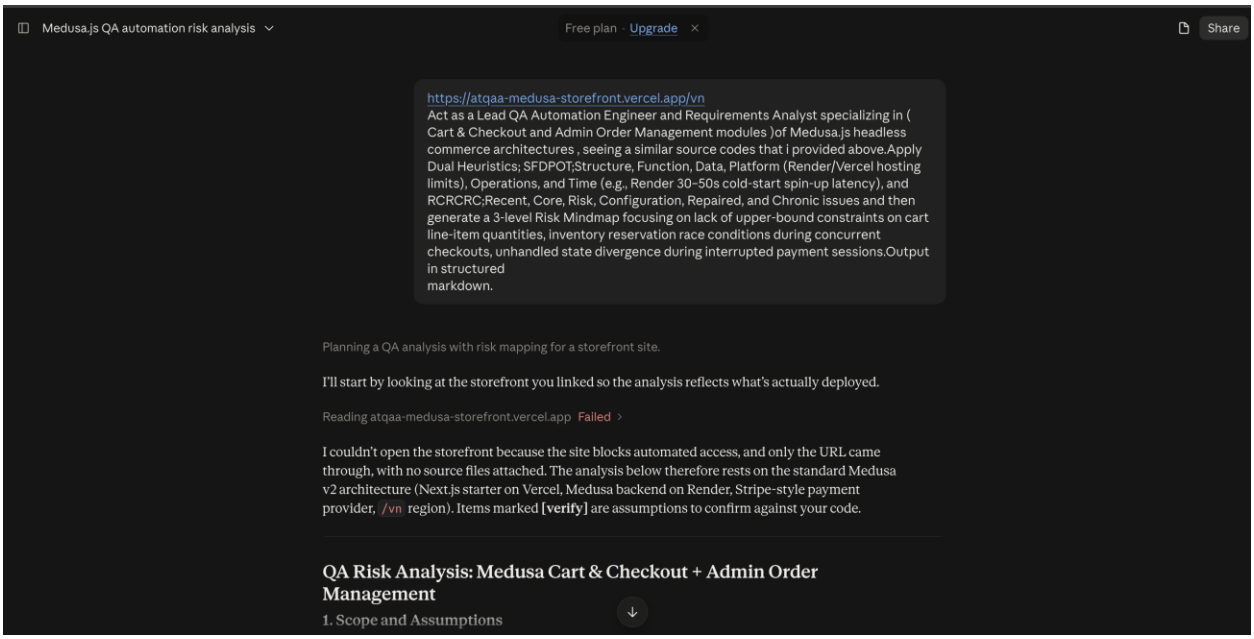


Management

1. Scope and Assumptions

Item	Assumption
Storefront	Next.js starter on Vercel (`/vn` = country-code route, VN region)
Backend / Admin	Medusa v2 on Render (likely free or starter tier)
Modules in scope	Cart, Inventory (reservations), Payment (collections/sessions), Order, Fulfillment, Admin
Key workflows	`addToCart`, `updateLineItem`, `completeCartWorkflow`, order capture/cancel/refund
Currency	VND, a zero-decimal currency **[verify]**

2. SFDPOT Analysis

S: Structure

- **Storefront → Backend:** the Next.js server actions call the Medusa Store API. The cart ID lives in a cookie.
- **Cart → Payment Collection → Payment Session → Order** is a multi-entity chain with no single DB transaction spanning it.
- **Inventory:** `InventoryItem` → `InventoryLevel` (stocked, reserved) → `ReservationItem`, linked to variants through `manage_inventory` and stock locations.
- **Admin:** Order → Fulfillment → Shipment, plus Payment → Capture/Refund, plus Order Change (edits, claims, returns).
- **Risk:** the cart and the order are separate aggr[illegible]s, so state can diverge whenever one step

- **Risk:** the cart and the order are separate aggregates, so state can diverge whenever one step succeeds and its neighbour fails.

F: Function

- Add, update and remove line items; apply promo codes; set shipping and billing addresses; choose a shipping option; initiate a payment session; complete the cart.
- Admin: view, cancel, fulfil, capture, refund, edit orders, and manage reservations.
- **Key checks:**
  - Quantity validation on add and on update (are the two paths consistent?).
  - Inventory check at add time versus at complete time.
  - Order totals match the cart totals match the payment amount.

D: Data

Data concern	Test focus
Quantity	0, negative, decimal, `2^31-1`, very large strings, non-numeric
Inventory	Stocked = 1 with N concurrent buyers; `allow_backorder` true or false; `manage_inventory` off
Money	VND zero-decimal rounding, promo and tax rounding, amount sent to the provider versus the order total
Identity	Guest versus logged-in cart merge, stale cart cookie, a cart already completed
Region	A cart created in `/vn` and then switched to another region

P: Platform (Vercel / Render limits)

- Render cold start (30–50s on free tier):
  - The storefront request may time out while the backend wakes.

P: Platform (Vercel / Render limits)

- **Render cold start (30–50s on free tier):**
  - The storefront request may time out while the backend wakes.
  - Users double-click "Place order" or retry, which produces duplicate `complete` calls.
  - A payment webhook arrives while the backend is asleep, so the provider retries, which can cause delayed or out-of-order processing.
- **Vercel serverless limits:** function duration caps (10–60s depending on plan), no persistent in-memory state, and server-action retries on timeouts.
- **Database connections:** a serverless storefront fanning out to a small Postgres on Render can exhaust the connection pool under concurrency.
- **Redis:** if event bus, locking or cache modules fall back to in-memory on Render, distributed locks and workflow retries are not safe across instances **[verify]**.
- **Browsers:** back/forward cache and multi-tab checkout, and mobile Safari redirects out to 3-D Secure.

O: Operations

- Admin cancels an order while a capture or fulfilment is in flight.
- Support manually edits inventory levels while customers have reservations open.
- Partial refunds and partial captures with VND rounding.
- Deployment during live carts: do schema or workflow changes break in-flight carts?
- Observability: are failed workflows and compensations logged and alertable?

T: Time

- **Cold-start latency** (above) that stretches the checkout critical path.
- **Reservation lifetime:** the unpaid reservation or cart expiry window. Medusa does not hold stock at add-to-cart, so the real contention window is complete-time **[verify]**.
- **Payment session expiry:** provider authorisation windows versus how long the cart persists.
- **Webhook ordering and retries:** `payment_int` `succeeded` arriving before or after the storefront's `complete` call.

Free plan · Upgrade ×

T: Time

- **Cold-start latency** (above) that stretches the checkout critical path.
- **Reservation lifetime:** the unpaid reservation or cart expiry window. Medusa does not hold stock at add-to-cart, so the real contention window is complete-time **[verify]**.
- **Payment session expiry:** provider authorisation windows versus how long the cart persists.
- **Webhook ordering and retries:** `payment_intent.succeeded` arriving before or after the storefront's `complete` call.
- **Time zones:** UTC-stored timestamps displayed in Asia/Saigon (UTC+7) in the admin; sale and promo windows crossing midnight.

3. RCRCRC Analysis

Heuristic	Candidate areas	Why it matters
Recent	Medusa v1 → v2 migration code; new Workflows SDK customisations; Vercel/Render redeploys; any new payment provider config	New code and config have the least test history.
Core	Add to cart → address → shipping → payment → complete; admin capture and fulfil	The revenue path. A defect here is a P1.
Risk	Uncapped quantity; concurrent checkout oversell; payment interruption divergence; webhook idempotency	Highest business and financial impact (see mindmap).
Configuration	`manage_inventory`, `allow_backorder`, stock locations and sales-channel mapping, region and currency, payment provider keys, CORS, Redis/locking, `DATABASE_URL` pool size, cookie settings	Misconfiguration silently disables safeguards.
Repaired	Any past fixes to cart totals, tax or promo calculation, inventory sync, payment capture	Fixes regress easily. Re-run their original reproduction cases.
Chronic	Stale carts, orphaned payment sessions, reservations never released, cart/order total mismatches, admin and storefront status drift	These recur because they come from the architecture, not from one bug.

4. 3-Level Risk Mindmap

```text
Cart & Checkout / Admin Order Mgmt: Risk Root
|
├── R1. Missing upper bound on line-item quantity
|   ├── R1.1 API / validation gap
|   |   ├── Store API accepts huge integer (e.g., 999999999) or int overflow
|   |   ├── `add` and `update` paths validate differently (inconsistent bounds)
|   |   └── Cart accepts a quantity greater than stock when backorder is on or invento
|   ├── R1.2 Business abuse / financial exposure
|   |   ├── Inventory hoarding or reservation exhaustion by one cart (denial of stock)
|   |   ├── Bulk-quantity promo or discount stacking, with total overflow or rounding
|   |   └── Shipping and weight calculation breaks at extreme quantities
|   └── R1.3 Performance / platform impact
|       ├── Giant totals trigger slow queries and serverless timeouts (Vercel)
|       ├── Cold-start Render plus heavy cart recalculation compounds latency
|       └── Admin order page struggles to render or export huge-quantity orders
|
├── R2. Inventory reservation race during concurrent checkouts
|   ├── R2.1 Check-then-reserve gap (TOC
|   |   └── Two carts pass the availability check for the last unit before either rese
```

```
|
├── R2. Inventory reservation ra                                  checkouts
|   ├── R2.1 Check-then-reserve gap (TOCTOU)
|   |   ├── Two carts pass the availability check for the last unit before either re
|   |   ├── Add-to-cart check is stale by the time of `complete`
|   |   └── Reservation and stocked/reserved counters updated non-atomically
|   ├── R2.2 Distributed-system / platform factors
|   |   ├── Locking module or Redis not configured, or in-memory only, on multi-instan
|   |   ├── Serverless retry of `complete` produces duplicate reservations
|   |   └── DB pool exhaustion makes the compensation step fail, leaving a partial res
|   └── R2.3 Operational consequence
|       ├── Oversold stock leads to cancellations and refunds (admin load)
|       ├── Orphaned reservations never released, so stock shows sold out falsely
|       └── Manual admin inventory edit conflicts with in-flight reservations
|
└── R3. State divergence on interrupted payment sessions
    ├── R3.1 Client-side interruption
    |   ├── Tab closed, network drop or 3-D Secure abandonment after the provider auth
    |   ├── User retries and a new payment session is created for the same cart
    |   └── Double-click or back button re-submits `complete` (no idempotency key)
    ├── R3.2 Server / platform interruption
    |   ├── Render cold start (30–50s) causes timeout; payment succeeded but order not
    |   ├── Webhook arrives before or after `complete`, or is retried out of order
    |   └── Workflow fails mid-way and the compensation step is itself interrupted
    └── R3.3 Resulting divergence
        ├── Paid but no order ("ghost payment"): customer charged, admin sees nothing
        ├── Order exists but payment is authorised-not-captured or failed in the admin
        └── Cart shows incomplete while an order exists (duplicate order on retry)
```

5. Priority Test Charters

Medusa.js QA automation risk analysis
Free plan · Upgrade ×
Share

5. Priority Test Charters

#	Charter	Risk	Technique	Priority
1	Send quantities 0, -1, 1.5, `2147483647`, `1e9`, "abc" to add and update endpoints and compare the responses	R1	API boundary and fuzzing	**P1**
2	Stock = 1; fire 20 concurrent `complete` calls with different carts (k6 or a script) and confirm exactly one order succeeds	R2	Concurrency / load	**P1**
3	Same cart, two parallel `complete` calls (double-click simulation) and confirm no duplicate order or reservation	R2, R3	Idempotency test	**P1**
4	Let Render sleep, then run checkout and trigger a timeout after provider authorisation. Verify order creation, webhook recovery and a reconciliation path	R3	Fault injection, cold-start	**P1**
5	Abandon 3-D Secure, return later, then re-pay. Check session reuse, amount integrity and no orphaned authorisation	R3	Exploratory	**P1**
6	Replay and reorder payment webhooks (succeeded before `complete`; duplicates)	R3	Event-order testing	**P2**
7	Cancel or fulfil in the admin while capture is in flight. Check status consistency across payment, fulfilment and order	R3	Admin concurrency	**P2**
8	VND rounding: promo, tax and multi-quantity totals versus the amount sent to the provider and the order total	R1, R3	Data / oracle comparison	**P2**
9	Release check for reservations: abandon car d confirm stock returns, with no phantom "sold out"	R2	Long-run / time-based	**P2**

Medusa.js QA automation risk analysis

	versus the amount sent to the pro total	Free plan · Upgrade ×	R3	comparison
9	Release check for reservations: abandon carts and confirm stock returns, with no phantom "sold out"		R2	Long-run / time-based	**P2**
10	Config audit: `manage_inventory`, `allow_backorder`, Redis and locking, and pool size per environment		All	Configuration review	**P2**

6. Suggested Mitigations to Verify

- **Quantity cap:** enforce a server-side maximum (per variant or per cart) in a workflow hook or validator, not only in the UI.
- **Reservations:** run the reserve step inside the cart-lock workflow and confirm the locking module uses Redis in production.
- **Idempotency:** use idempotency keys on `complete` and make the order creation keyed to the cart ID.
- **Payment reconciliation:** add a scheduled job that matches captured or authorised payments with no order, and carts with orders in the wrong state.
- **Cold start:** keep the backend warm (paid tier or a keep-alive ping), and give the storefront a "payment in progress" state rather than a blind retry.

If you share the source files (or a GitHub repo), I can replace the **[verify]** items with confirmed findings and tailor the test cases to your actual workflows. I can also turn this into a downloadable file such as a Word or Excel test plan.

Claude works directly with your codebase Install ×

21 minutes ago